TECHNICAL NOTE

git-unlocked: An Open-Source Modular Curriculum for Git and Version Control Education

Isaac Adjei

College of Engineering and Physical Sciences, Aston University, Birmingham, United Kingdom

ORCID 0009-0001-8298-5098 | academic@isaacadjei.me

October 2026 | Describes git-unlocked release 1.2.6

Abstract. Git is the dominant version control system in software development, yet most developers learn it on their own. Freely available material tends either to stop at a handful of basic commands or to cover a single hosting platform. This note describes git-unlocked, an openly licensed curriculum that takes a learner from no prior knowledge to professional practice with Git. The course comprises 182 Markdown files in 12 sections, roughly 437,000 words of prose and an estimated 79 hours of guided study. It covers Git itself, seven hosting platforms, editor and terminal integration, scenario-based team practice and a reference set. Content files share a common structure with a difficulty label, an estimated study time, worked commands for Windows, macOS and Linux and, in most files, exercises and a section on common mistakes. The note situates the course within research on how Git is learned and taught, sets out the instructional design principles behind it and describes its structure and maintenance. It closes with the course's limitations and a plan for evaluating it with learners. The course is released under the Creative Commons Attribution-ShareAlike 4.0 International licence and every release is archived with a DOI.

Keywords: version control; Git; computing education; open educational resources; software engineering education; curriculum design

1 Introduction

Version control is a foundational skill for anyone who writes software. In the 2022 Stack Overflow Developer Survey, 93.87% of 71,379 respondents reported using Git, while 17% of respondents who were still learning to code reported using no version control system at all [29]. Version control has also long been recommended as a basic practice for researchers who write code [3, 35].

Git is nevertheless difficult to learn. Analyses of its conceptual design have traced common confusions to concepts that fail to match the purposes users bring to them [26, 27]. A study of 80,370 Git questions on Stack Overflow found that developers continue to struggle with a range of commands and that self-directed learning is the main way they learn Git in practice [36]. When learning is largely self-directed, the quality and coverage of freely available material matters.

The course began as my personal reference and grew into git-unlocked, a free and openly licensed course written to provide a single structured route from first principles to professional practice. Its contributions are:

1. a complete curriculum of 182 files covering Git, seven hosting platforms, tooling, team scenarios and reference material, with a consistent structure throughout;

2. an explicit instructional design, described in Section 3, that combines worked examples, labelled progression, cross-platform commands and a focus on common mistakes;
3. an open, versioned and citable form: the course is maintained in a public repository with automated checks and each release is archived with a DOI.

Section 2 reviews research on learning and teaching Git together with existing open resources. Section 3 sets out the design principles. Sections 4 to 6 describe the curriculum, its accessibility and its maintenance. Section 7 covers licensing and adoption, Section 8 the limitations and Section 9 a plan for evaluation.

2 Background and related work

2.1 Why Git is difficult to learn

Perez De Rosso and Jackson analysed Git's design in terms of its concepts and the purposes they serve and identified mismatches that make the system hard to use even for experienced developers [26]. Their later work catalogued these mismatches as misfits, related them to questions asked on Stack Overflow and proposed a redesign [27]. From a teaching perspective, Isomöttönen and Cochez reported the challenges and confusions students met when learning Git, drawing on survey data from a project-based course and on teaching experience across several courses [18]. Yang et al. identified the commands most often associated with difficult questions and found that developers mostly teach themselves [36]. Together these findings suggest that learners need conceptual explanation alongside commands and that guidance on recovering from mistakes is particularly valuable.

2.2 Teaching version control in courses

Version control has been introduced into courses in several ways. Lawrance et al. described adopting it in a first programming course for engineering students outside computer science, supported by hosting services [23]. Haaranen and Lehtinen used Git as the platform through which students worked on and submitted coursework, so that version control was practised throughout a course [15]. Zagalsky et al. examined how GitHub was being used as a collaborative platform for education [37]. A survey of 7,530 students and 300 educators by Hsing and Gennarelli found that the use of GitHub in classrooms was associated with differences in students' reported learning outcomes and experiences [17]. Glassey reviewed the tools that support adopting Git and GitHub in teaching and noted the technical barrier that adoption still presents [14].

More recent work has broadened the disciplines and methods involved. Beckman et al. described how Git was made an explicit learning objective in statistics and data science courses at several institutions [2]. Jabrayilzade et al. ran a four-session Git and GitHub training programme for 258 students and reported that the proportion who considered themselves insufficiently prepared fell from 67% to 9% [19]. Gilson et al. designed a student-led course covering version control and other practical tools to reduce the gap between academic study and industry practice [10]. Chen et al. evaluated a serious game for teaching Git concepts and reported improved motivation [5]. Most recently, Wagner and Thurner argued that teaching should separate conceptual understanding of the Git model from the detail of individual commands and proposed reordering course content to begin with branches [33].

2.3 Open learning resources

Several high-quality free resources already exist. *Pro Git* [4] is the most complete reference to Git itself but does not cover the hosting platforms most teams work through. The Software Carpentry lesson *Version Control with Git* [30] and the version control lecture of MIT's *Missing Semester* [25] are well-regarded introductions designed for a session or two. Workshop curricula built on Carpentries material have also been published with Git as one part of a wider set of research skills [22]. Interactive resources such as GitHub Skills [12] and Learn Git Branching [8] teach through practice within a narrower scope. Platform documentation [11, 13] and tutorials such as Atlassian's [1] are thorough for their own products but assume the reader already understands Git.

More broadly, open educational resources allow material to be reused and adapted without cost, a principle set out in UNESCO's Recommendation on Open Educational Resources [32]. A review of studies in higher education found that adopting open resources generally did not reduce student learning outcomes [16].

2.4 The gap

Existing resources are strong individually but leave a self-directed learner to assemble a path between them: from Git's concepts to its commands, from one platform to another and from individual use to work in a team. git-unlocked aims to provide that path in one place under an open licence, with consistent structure and level labelling. It links out to the primary sources above rather than replacing them.

3 Design principles

Concepts before commands. The introduction explains what version control is and how Git models history before any command is used. Conceptual files also precede the commands that depend on them. This follows the recommendation to treat conceptual understanding and applied skill as distinct learning goals [33]. Unlike the ordering Wagner and Thurner propose, git-unlocked keeps the conventional sequence of committing before branching, which matches most of the platform documentation that learners will meet afterwards.

Guided instruction with worked examples. Novices learn more effectively from explicit guidance and worked examples than from unguided discovery [20]. Conventional problem solving can impose a cognitive load that interferes with learning [31]. Each lesson therefore explains a command and shows it in a worked example, often with its expected output, before asking the learner to apply it in a *Try It Yourself* exercise.

Labelled progression. Each file is labelled beginner, intermediate or advanced. The levels broadly follow the progression of the revised Bloom's taxonomy [21]. Beginner files concentrate on remembering and understanding core concepts and applying basic commands. Intermediate files apply and combine them in workflows. Advanced files and real-world scenarios ask the learner to analyse and evaluate situations such as a broken history. Labelling at file level lets a learner follow the beginner path across the whole course before returning for depth.

Mistakes as content. Many Git difficulties arise when something has already gone wrong [27, 36]. Most lessons therefore end with a *Common Mistakes* section. The reference section collects these into a single guide alongside a disaster-recovery scenario.

Distinct kinds of documentation. Following the distinction drawn by the Diátaxis framework

between tutorials, how-to guides, reference and explanation [28], the course separates guided lessons from scenario-based guides and from reference material such as the cheat sheet, glossary and per-platform feature references.

Every operating system a learner uses. Commands that differ between operating systems are shown for Windows and for macOS and Linux, each labelled, so that no learner is excluded by the computer they own.

4 Curriculum structure

4.1 Sections and pathway

The course is organised into 12 numbered sections (Table 1). Figure 1 shows the intended pathway: an introduction, the core Git section, the hosting platforms, tooling and real-world practice, with reference material and a first-contribution exercise available throughout. Each section can also be read on its own, so an educator can adopt one section without the rest.

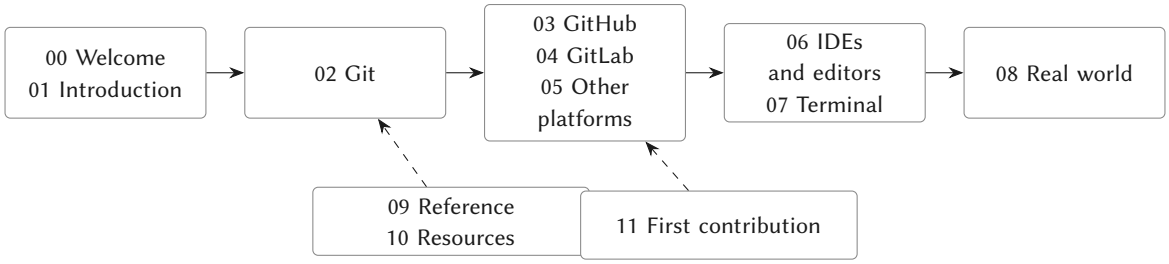


Figure 1: The intended pathway through git-unlocked. Solid arrows show the main sequence; dashed arrows show material used alongside it.

Table 1: Sections of git-unlocked with the number of content files and the estimated guided study time in each.

Section	Content	Files	Hours
00 Welcome	Purpose, audience and learning outcomes	1	0.1
01 Introduction	How to use the course, setting up and version control concepts	3	1.0
02 Git	Core concepts to internals: branching, merging, conflicts, rebasing, history rewriting, hooks, bisect, signing, large repositories and a command reference	29	10.5
03 GitHub	Repositories, pull requests, Actions, Pages, Projects, security features, the CLI and a features reference	28	12.7
04 GitLab	Merge requests, CI/CD, planning, security, self-hosting and the API	16	9.0
05 Other platforms	Bitbucket, Azure DevOps, Gitea, Forgejo and Codeberg	62	28.5
06 IDEs and editors	Git in eleven editors and IDEs, with notes on others	14	6.6
07 Terminal	Shells, configuration, aliases, safety and tools such as lazygit, delta and fzf	14	5.2
08 Real world	Scenarios: open source, teams, GitOps, monorepos, migration and disaster recovery	8	4.2
09 Reference	Cheat sheet, glossary of 98 terms, common mistakes and security	4	1.0
10 Resources	A curated index of 142 external resources by type and level	1	n/a
11 First contribution	A sandbox for making a first pull request	2	0.2
Total		182	78.8

Hours are the sum of each file's stated study time. The 13 reference files marked "use as needed" are not included.

4.2 Anatomy of a file

Apart from the contributors list, every file opens with a difficulty label and an estimated study time, and almost all continue with a table of contents. Lesson files then share a common set of closing sections, summarised in Table 2. The files without exercises are overviews, reference pages and the welcome and setup pages, where exercises would not fit. Across the course there are 3,686 code blocks.

Table 2: Recurring elements and the number of the 182 files that contain each.

Element	Purpose	Files
Difficulty and study time	Lets a learner choose a path and plan time	181
Table of contents	Navigation within long files	173
Sources	Links to the primary documentation behind the file	165
Summary	The key points of the file	155
Try It Yourself	Hands-on exercises applying the commands shown	152
Common Mistakes	Typical errors on the topic and how to avoid or recover from them	152

4.3 Progression

Table 3 shows how difficulty is distributed. The Git section leans towards advanced material because it runs from first commands to Git's internals, while the platform sections are mostly beginner and intermediate. The real-world section and the reference material are labelled as spanning levels, since they are used at different stages.

Table 3: Difficulty labels by section. "Mixed" covers files labelled as spanning more than one level.

Section	Beginner	Intermediate	Advanced	Mixed
00 to 01 Welcome and introduction	4	0	0	0
02 Git	6	8	14	1
03 GitHub	12	7	2	7
04 GitLab	5	7	4	0
05 Other platforms	27	27	8	0
06 IDEs and editors	5	7	2	0
07 Terminal	7	7	0	0
08 Real world	0	0	0	8
09 to 11 Reference, resources and first contribution	1	0	0	5
Total	67	63	30	21

One file, the contributors list, carries no label.

4.4 Platforms and tools

The platform sections cover GitHub (28 files), GitLab (16) and five further platforms: Azure DevOps (15), Bitbucket (14), Gitea (13), Forgejo (10) and Codeberg (10). Each platform section opens with an overview, continues with account setup and the platform's own features and ends with a feature reference, so that a learner who moves between platforms meets the same structure each time. The tooling sections cover Git inside eleven editors and IDEs and in the terminal, including widely used tools such as lazygit, delta and fzf.

4.5 Real-world practice and safety

The real-world section is built around scenarios rather than commands: contributing to an open-source project, working in a team, adopting GitOps, managing a monorepo, migrating between platforms and recovering from disasters such as a lost branch or an exposed secret. The reference section includes a security guide covering secret detection, signed commits and supply-chain risks, drawing on documented incidents.

5 Accessibility

The course is written in plain Markdown, which renders as structured HTML on GitHub and remains readable in any text editor or screen reader. Every lesson uses a heading hierarchy with a table of contents. Warnings and tips use GitHub's alert syntax, which presents them as labelled callouts rather than relying on colour alone. Emoji serve as compact markers, mainly for difficulty, operating system and support in comparison tables. On each file's difficulty line the marker is paired with its text label, while some headings use the marker alone. How well these markers read with screen readers is one of the questions for the review described below. Commands are given as text in code blocks rather than as screenshots, so they can be copied, magnified and read aloud. A formal accessibility review has not yet been carried out. The Web Content Accessibility Guidelines [34] would be the reference for that review and for any hosted version of the course.

6 Quality assurance, versioning and maintenance

The course is maintained as a public Git repository, so it demonstrates the practices it teaches. Two automated checks run in continuous integration. Every push and pull request is linted with markdownlint [9] against a shared configuration, which keeps the structure of all files consistent. Every pull request and a scheduled weekly run check all external links with lychee [24], so that broken references are found as the web changes. Dependency updates for these workflows are automated.

Releases follow semantic versioning and each is recorded in a changelog. A citation file in the Citation File Format [6] gives the preferred citation. Every tagged release is archived automatically on Zenodo with its own DOI. Contributions follow a documented template that fixes the structure of each new file. The repository also carries a contributing guide, a code of conduct, a security policy, a roadmap and issue forms. At release 1.2.6 the history comprises 199 commits since March 2026.

7 Licensing and adoption

The course is released under the Creative Commons Attribution-ShareAlike 4.0 International licence [7]. Anyone may copy, adapt and redistribute the material, including for teaching, provided the source is credited and adaptations are shared under the same terms. This suits written course material: an educator can adapt a single section to their own module and publish the result, while improvements remain open to others.

The course supports three main uses:

- **Independent learning**: a learner starts at the welcome page, follows the beginner-labelled files across the course and returns for intermediate and advanced material as needed.
- **Supplementary teaching material**: an educator assigns particular sections alongside a programming, data science or software engineering module and uses the exercises as practical work, in the

way several of the courses reviewed in Section 2 integrate Git into existing teaching [2, 15].

- **Reference**: the command reference, cheat sheet, glossary and common-mistakes guide serve working developers.

The first-contribution sandbox offers a low-risk way to practise the full fork, branch and pull request workflow on a real repository.

8 Limitations

The course has not yet been evaluated with learners in a structured way, so no claims are made here about learning outcomes. The design principles in Section 3 describe intent rather than measured effect. The course is written and maintained by a single author, which limits the range of perspectives and the capacity for review. Its breadth means that some platform sections describe features that change frequently and will need regular updating. The material is text-based and has no interactive quizzes or diagrams. The estimated study times are my own estimates and have not been measured. Finally, the conventional ordering of content was chosen for consistency with platform documentation, and alternative orderings such as the one proposed by Wagner and Thurner [33] have not been compared.

9 Evaluation plan and future work

A structured evaluation would allow the effect of the course to be measured rather than assumed. A suitable first study would follow the approach used by Jabrayilzade et al. [19]. Learners would complete a short self-efficacy questionnaire and a set of practical tasks before and after working through the beginner pathway, with the tasks drawn from the difficulties identified in the literature [18, 36]. Recording which files learners use, which exercises they complete and where they stop would show whether the labelled pathway works as intended. A study of this kind would require ethical approval and informed consent from participants.

Planned development, recorded in the project roadmap, includes interactive quiz pages with immediate feedback, animated diagrams of key Git concepts, a hosted website version of the course, an audit of internal cross-references, a formal accessibility review and a preconfigured development container.

10 Conclusion

git-unlocked offers a free, openly licensed and citable route through Git, from its concepts to professional team practice on the platforms learners are likely to use. Its consistent structure, labelled progression, cross-platform commands and emphasis on mistakes draw on what is known about how Git is learned. Its open licence lets educators adapt any part of it. The most important next step is an evaluation with learners, so that the design can be tested and improved on evidence.

Availability

Repository	`https://github.com/zaccesss/git-unlocked`
Course archive	doi:10.5281/zenodo.20695630 (all releases)
Licence	Creative Commons Attribution-ShareAlike 4.0 International
Format	Markdown, readable on GitHub or in any text editor

Funding

This work received no external funding.

Competing interests

The author declares no competing interests.

References

[1] Atlassian. *Git tutorials*. https://www.atlassian.com/git/tutorials. Accessed 15 October 2026.

[2] M. D. Beckman, M. Çetinkaya-Rundel, N. J. Horton, C. W. Rundel, A. J. Sullivan and M. Tackett. Implementing version control with Git and GitHub as a learning objective in statistics and data science courses. *Journal of Statistics and Data Science Education*, 29(sup1):S132 to S144, 2021. doi:10.1080/10691898.2020.1848485.

[3] J. D. Blischak, E. R. Davenport and G. Wilson. A quick introduction to version control with Git and GitHub. *PLOS Computational Biology*, 12(1):e1004668, 2016. doi:10.1371/journal.pcbi.1004668.

[4] S. Chacon and B. Straub. *Pro Git*. Apress, 2nd edition, 2014. doi:10.1007/978-1-4842-0076-6. Online edition: https://git-scm.com/book/en/v2.

[5] H.-M. Chen, B.-A. Nguyen, Y.-W. Chang and C.-R. Dow. A gamified method for teaching version control concepts in programming courses using the Git Education Game. *Electronics*, 13(24):4956, 2024. doi:10.3390/electronics13244956.

[6] Citation File Format. https://citation-file-format.github.io/. Accessed 15 October 2026.

[7] Creative Commons. *Attribution-ShareAlike 4.0 International*. https://creativecommons.org/licenses/by-sa/4.0/.

[8] P. Cottle. *Learn Git Branching*. https://learngitbranching.js.org. Accessed 15 October 2026.

[9] D. Anson. *markdownlint-cli2*. https://github.com/DavidAnson/markdownlint-cli2. Accessed 15 October 2026.

[10] G. Gilson, S. Ott, N. R. Ledesma, A. Prabhu and J. Porquet-Lupine. Design and evaluation of "The Missing CS Class", a student-led undergraduate course to reduce the academia-industry gap. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education (SIGCSE)*, pages 467 to 473, 2022. doi:10.1145/3478431.3499422.

[11] GitHub. *GitHub Docs*. https://docs.github.com. Accessed 15 October 2026.

[12] GitHub. *GitHub Skills*. https://skills.github.com. Accessed 15 October 2026.

[13] GitLab. *GitLab Docs*. https://docs.gitlab.com. Accessed 15 October 2026.

[14] R. Glassey. Adopting Git/GitHub within teaching: a survey of tool support. In *Proceedings of the ACM Conference on Global Computing Education (CompEd)*, pages 143 to 149, 2019. doi:10.1145/3300115.3309518.

[15] L. Haaranen and T. Lehtinen. Teaching Git on the side: version control system as a course platform. In *Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE)*, pages 87 to 92, 2015. doi:10.1145/2729094.2742608.

[16] J. Hilton. Open educational resources and college textbook choices: a review of research on efficacy and perceptions. *Educational Technology Research and Development*, 64(4):573 to 590, 2016. doi:10.1007/s11423-016-9434-9.

[17] C. Hsing and V. Gennarelli. Using GitHub in the classroom predicts student learning outcomes and classroom experiences: findings from a survey of students and teachers. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE)*, pages 672 to 678, 2019. doi:10.1145/3287324.3287460.

[18] V. Isomöttönen and M. Cochez. Challenges and confusions in learning version control with Git. In *Information and Communication Technologies in Education, Research and Industrial Applications (ICTERI 2014)*,

Communications in Computer and Information Science, volume 469, pages 178 to 193. Springer, 2014. doi:10.1007/978-3-319-13206-8_9.

[19] E. Jabrayilzade, F. S. Uyanık, E. Sülün and E. Tüzün. An interactive approach to teaching Git version control system. In *Proceedings of the 55th Hawaii International Conference on System Sciences (HICSS)*, 2022. doi:10.24251/HICSS.2022.112.

[20] P. A. Kirschner, J. Sweller and R. E. Clark. Why minimal guidance during instruction does not work: an analysis of the failure of constructivist, discovery, problem-based, experiential and inquiry-based teaching. *Educational Psychologist*, 41(2):75 to 86, 2006. doi:10.1207/s15326985ep4102_1.

[21] D. R. Krathwohl. A revision of Bloom's taxonomy: an overview. *Theory Into Practice*, 41(4):212 to 218, 2002. doi:10.1207/s15430421tip4104_2.

[22] Z. Lapp, K. L. Sovacool, N. Lesniak, D. King, C. Barnier, M. Flickinger et al. Developing and deploying an integrated workshop curriculum teaching computational skills for reproducible research. *Journal of Open Source Education*, 5(47):144, 2022. doi:10.21105/jose.00144.

[23] J. Lawrance, S. Jung and C. Wiseman. Git on the cloud in the classroom. In *Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE)*, pages 639 to 644, 2013. doi:10.1145/2445196.2445386.

[24] lychee contributors. *lychee: a fast link checker*. https://github.com/lycheeverse/lychee. Accessed 15 October 2026.

[25] A. Athalye, J. Gjengset and J. J. Gonzalez Ortiz. Version control (Git). In *The Missing Semester of Your CS Education*. MIT, 2020. https://missing.csail.mit.edu/2020/version-control/.

[26] S. Perez De Rosso and D. Jackson. What's wrong with Git? A conceptual design analysis. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms and Reflections on Programming and Software (Onward!)*, pages 37 to 52, 2013. doi:10.1145/2509578.2509584.

[27] S. Perez De Rosso and D. Jackson. Purposes, concepts, misfits and a redesign of Git. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*, pages 292 to 310, 2016. doi:10.1145/2983990.2984018.

[28] D. Procida. *Diátaxis: a systematic approach to technical documentation authoring*. https://diataxis.fr/. Accessed 15 October 2026.

[29] Stack Overflow. *2022 Developer Survey: version control systems*. https://survey.stackoverflow.co/2022/#version-control-version-control-system.

[30] Software Carpentry. *Version Control with Git*. https://swcarpentry.github.io/git-novice/. Accessed 15 October 2026.

[31] J. Sweller. Cognitive load during problem solving: effects on learning. *Cognitive Science*, 12(2):257 to 285, 1988. doi:10.1207/s15516709cog1202_4.

[32] UNESCO. *Recommendation on Open Educational Resources (OER)*. 2019. https://www.unesco.org/en/legal-affairs/recommendation-open-educational-resources-oer.

[33] G. Wagner and L. Thurner. Rethinking how we teach Git: pedagogical recommendations and practical strategies for the information systems curriculum. *Journal of Information Systems Education*, 36(1):1 to 12, 2025. doi:10.62273/BTKM5634.

[34] W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 12 December 2024. https://www.w3.org/TR/WCAG22/.

[35] G. Wilson, D. A. Aruliah, C. T. Brown, N. P. Chue Hong, M. Davis, R. T. Guy, S. H. D. Haddock, K. D. Huff, I. M. Mitchell, M. D. Plumbley, B. Waugh, E. P. White and P. Wilson. Best practices for scientific computing. *PLOS Biology*, 12(1):e1001745, 2014. doi:10.1371/journal.pbio.1001745.

[36] W. Yang, C. Zhang, M. Pan, C. Xu, Y. Zhou and Z. Huang. Do developers really know how to use Git commands? A large-scale study using Stack Overflow. *ACM Transactions on Software Engineering and Methodology*, 31(3):1 to 29, 2022. doi:10.1145/3494518.

[37] A. Zagalsky, J. Feliciano, M.-A. Storey, Y. Zhao and W. Wang. The emergence of GitHub as a collaborative

platform for education. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work and Social Computing (CSCW)*, pages 1906 to 1917, 2015. doi:10.1145/2675133.2675284.